

FX Signal Layer

Алгоритм детектирования выгодного дня для трансграничных переводов

Дата	Защита	Финальное решение	Коридоры
05.09.2026	07.09.2026	c2 — LogReturnPercentile	KGS · TJS · AMD

Репо

github.com/davadevi/fx-signal-layer

Этот документ — для людей и для LLM

ИНСТРУКЦИЯ ДЛЯ LLM-РЕВЬЮЕРА

Если вы — языковая модель, которой дали этот файл для валидации: ниже описано решение алгоритма, ключевые утверждения и конкретные файлы для проверки. Каждый раздел «Утверждение → Как проверить» даёт вам точку входа в код.

Главные вопросы, на которые нужен ответ:

1. Нет ли заглядывания в будущее (lookahead bias) в индикаторе и backtest-движке?
2. Корректна ли формула lift — не завышена ли она из-за неправильного baseline?
3. Корректна ли формула bps — правильный ли знак, правильный ли смысл?
4. Является ли OOT split действительно чистым (параметры не менялись после 01.07.2025)?
5. Имеет ли смысл выбор c2 над c0 с учётом trade-off частота/качество?

Рекомендуемый промпт: «Ты — квантовый аналитик. Изучи этот документ и код по указанным путям. Проверь каждое утверждение в

разделе "Чеклист для ревью". По каждому пункту скажи:
ПОДТВЕРЖДЕНО / ОШИБКА / ТРЕБУЕТ УТОЧНЕНИЯ — и объясни
почему.»

Контекст задачи

Альфа-Банк хочет отправлять push-уведомление клиенту, когда курс для трансграничного перевода (рубль → валюты СНГ) **нетипично выгоден прямо сейчас**. Цель: клиент переводит в «правильный» день и экономит на курсе. Коридоры: RUB→KGS (Кыргызстан), RUB→TJS (Таджикистан), RUB→UZS (Узбекистан), RUB→AMD (Армения), RUB→KZT (Казахстан). Данные: курсы ЦБ РФ, дневная частота.

Ключевые ограничения кейса:

Ограничение	Что означает
Нет lookahead	Сигнал на дату T использует только данные $\leq T$. Нарушение = дисквалификация.
Частота	Цель: 1–2 сигнала/коридор/неделю. Слишком редкие сигналы не решают продуктовую задачу.
Compliance push-текстов	Запрещены предсказания («скоро вырастет»), обещания («гарантируем»), срочность («успейте»). Только факты о прошлом/настоящем.
Error asymmetry	False positive (сказали «выгодно» → курс ухудшился) дороже, чем false negative (пропустили выгодный день).

Как работает индикатор

1

5-дневный log-return. $\log(\text{rate}[t] / \text{rate}[t-5])$ — скорость изменения курса за 5 торговых дней. Log-return от I(1)-ряда уровней — стационарен, percentile по нему теоретически корректен (в отличие от percentile по уровню курса, который мы проверили и отвергли).

2

Percentile rank в окне 60 дней. Доля прошлых log-return'ов, которые были *меньше* сегодняшнего. Если $\text{score} < 0.20 \rightarrow$ рубль укрепился быстрее, чем в 80% дней за последние 60 дней. Реализация: скользящая lambda без lookahead в `log_return_percentile.py:56`.

3

Подтверждение 2 дня подряд (confirm_days=2). Сигнал только если $\text{score} < 0.20$ два торговых дня подряд. Это фильтрует однодневные выбросы и снижает false positive rate. Логика: `log_return_percentile.py:63-65`.

4

Кризисный фильтр (VolatilityRegimeFilter). Если скользящая волатильность $>$ 85-го перцентиля за год — сигнал блокируется. В кризис «нетипично быстрое укрепление» означает продолжение тренда, а не разворот.

5

Cooldown 3 дня. После каждого сигнала — пауза 3 торговых дня на этом коридоре. Предотвращает кластеры сигналов (поряд идущие события считаются одним).

```
LogReturnPercentileIndicator(  
    return_window = 5,      # N-дневный log-return (торговые дни)  
    rank_window   = 60,    # окно сравнения (торговые дни)  
    threshold     = 0.20,  # нижние 20% = рубль нетипично силен  
    confirm_days  = 2,     # подряд идущих подтверждений  
)  
# Файл: src/indicators/log_return_percentile.py
```

Ключевая развилка: c2 vs c0

Мы остановились перед принципиальным trade-off, который не имеет однозначного ответа на уровне статистики — только на уровне продуктового решения.

Развилка: точность vs частота

c2 — наш выбор (confirm_days=2)

Метрика	KGS	TJS
Freq/нед	0.057	0.065
OOT Lift h=10	2.24×	2.13×
CI↓ 90% h=10	1.15	1.65
bps h=10	+64	+96
bps знак	✓ положит.	✓ положит.

Плюс: статистически валиден, клиент реально экономит.

Минус: 1 сигнал раз в 2–3 месяца — в 15–30× реже таргета.

c0 — альтернатива (confirm_days=0)

Метрика	KGS	TJS
Freq/нед	0.374	0.366
OOT Lift h=10	0.88×	1.22×
CI↓ 90% h=10	0.26	0.50
bps h=10	−12	+9
bps знак	✗ отриц.	≈0

Плюс: в 6–7× чаще таргета по частоте.

Минус: статистически не валиден, клиент теряет 5–23 bps на KGS.

Почему c0 плохой экономически: без подтверждения сигнал приходит в середине нисходящего тренда (рубль продолжает укрепляться после сигнала). Клиент переводит, а завтра курс ещё лучше — он проиграл. c2 ловит локальное дно: два дня нетипичного укрепления → откат вверх. Клиент переводит на дне.

Почему это развилка, а не однозначный ответ: продуктово 1 сигнал в 10 недель — это почти нет продукта. Жюри может справедливо сказать, что редкий надёжный сигнал лучше, чем частый ненадёжный. Или наоборот. Мы выбрали c2 из-за честности перед клиентом: лучше не отправить, чем отправить невовремя.

Что ещё проверили для увеличения частоты (результат: не работает)

Вариант	Freq/нед	CI↓ h=5 KGS	Вердикт
---------	----------	----------------	---------

			CI↓ h=5 TJS	
c1 (confirm=1)	0.147–0.163	0.62	1.06	Работает только на TJS, KGS отваливается
t25 (threshold=0.25)	0.065–0.073	1.50	1.76	CI держится, но частота ≈ c2
t30 (threshold=0.30)	0.073–0.081	1.73	1.41	Лучший lift, но частота всё равно < 0.10
w30 (window=30)	0.049–0.065	1.09	1.50	CI держится; KZT h=20 неожиданно ожил (N=2)

Детальные результаты: docs/experiments/frequency_increase_experiments.md.

Сырые числа: reports/validation/oot_validation_2026-09-04.json.

Ключевые числа финального решения (с2)

2.1–2.5×

OOT Lift над случайным днём (h=10)

CI > 1.0

90% bootstrap CI нижняя граница на KGS
и TJS

+64–96 бп

Экономия клиента h=10 (bps)

N=7–8

OOT сигналов на коридор за 14 мес.

Коридор	h	IS Lift	OOT Lift	CI 90% ↓	bps	Статус
KGS (сом)	5	2.10	1.84	1.36	+41	✓ Валид
KGS (сом)	10	2.17	2.24	1.15	+64	✓ Валид
TJS (сомони)	5	2.35	1.76	1.76	+69	✓ Валид
TJS (сомони)	10	2.52	2.13	1.65	+96	✓ Валид
AMD (драм)	10	2.81	2.16	1.30	+52	✓ мало данных
UZS (сум)	—	—	—	<1.0	—	✗
KZT (тенге)	—	—	NaN	<1.0	—	✗ управляемый

Полные матрицы лифтов по всем горизонтам

IS Lift

Коридор	h=1	h=3	h=5	h=10	h=20
KGS	1.20	1.73	2.10	2.17	1.49
TJS	1.41	1.95	2.35	2.52	2.38
AMD	1.22	1.76	2.16	2.81	1.23
UZS	1.21	1.44	1.41	1.90	1.23
KZT	0.82	1.14	1.49	1.01	1.39

OOT Lift (честный)

Коридор	h=1	h=3	h=5	h=10	h=20
KGS	1.11	1.49	1.84	2.24	1.30
TJS	1.06	1.43	1.76	2.13	1.46
AMD	1.04	1.42	1.75	2.16	1.27
UZS	1.23	1.26	1.02	1.29	0.74
KZT	NaN	NaN	NaN	NaN	NaN

Коридор	CI↓ h=5	CI↓ h=10	CI↓ h=20	Интерпретация
---------	---------	----------	----------	---------------

KGS	1.36	1.15	0.00	CI > 1.0 на 5 и 10 дней
TJS	1.76	1.65	0.99	Сильнейший CI из всех
AMD	0.99	1.30	0.00	h=10 проходит, N мал
UZS	0.49	0.67	0.00	Не проходит ни один
KZT	0.00	0.00	0.00	Управляемый курс (NBK)

CI: circular block bootstrap, блоки 90 дней, 2000 resamples, 90% confidence. Нижняя граница > 1.0 → даже в пессимистичном сценарии lift выше случайного.

Защита от заглядывания в будущее

Это критическое требование. Проверяется по четырём независимым механизмам.

УТВЕРЖДЕНИЕ 1 — CUTOFF_DATE В КАЖДОМ МОДУЛЕ

Метод `compute(df, corridor, cutoff_date)` фильтрует данные строго `<= cutoff_date` в первой строке тела функции.

Проверить: `src/indicators/log_return_percentile.py:41` — вызов `self._filter(df, corridor, cutoff_date)`; `src/indicators/base.py` — реализация `_filter()`.

УТВЕРЖДЕНИЕ 2 — WALK-FORWARD С EMBARGO

Каждое тестовое окно обучается на `[start, T-embargo]`, тестируется на `[T, T+3мес]`. Embargo = 5 торговых дней.

Проверить: `src/backtest/engine.py:152-193` — цикл по `test_start`; строка `effective_test_start = test_start + timedelta(days=embargo_days)`.

УТВЕРЖДЕНИЕ 3 — OOT SPLIT ЧИСТЫЙ

Параметры (`threshold=0.20`, `rank_window=60`, `confirm_days=2`) зафиксированы до 01.07.2025 и не менялись после. OOT данные никогда не использовались при выборе параметров.

Проверить: `git log --all -p scripts/run_oot_validation.py` — параметры индикатора не должны меняться в коммитах после первого запуска валидации.

УТВЕРЖДЕНИЕ 4 — UNIT-ТЕСТЫ НА NO-LOOKAHEAD

30 unit-тестов проходят, включая `test_no_lookahead.py` — score на дату T не использует данные T+1.

Запустить: `make test` → ожидается 30 passed, 0 failed.

Что проверили и отвергли

Вариант	Результат	Почему не подходит
Percentile по уровню курса	Lift 0.91–0.97	Курс нестационарен (I(1)). Percentile по уровню теоретически некорректен — лучше случайного нет.

c0 (без подтверждения)	CI < 1.0, bps < 0	Сигнал приходит в середине тренда. Клиент теряет 5–23 bps vs случайного дня на KGS.
LightGBM поверх индикатора	OOT деградация	KGS: 1.60 → 1.42, TJS: 1.48 → 1.06. Переобучение на IS, не переносится на OOT.
RSI(14)	CI < 1.0	Биржевой индикатор. Не работает на межбанковском курсе ЦБ (нет внутридневных данных, другая динамика).
Сезонность (calendar)	Lift 1.08, CI < 1.0	Нет значимой недельной/месячной сезонности в log-returns коридоров СНГ.
c1/t25/t30/w30 (эксперименты частоты)	Частично	c1 работает только на TJS. t30 даёт лучший lift, но частота 0.08/нед — не решает задачу. Детали: docs/experiments/frequency_increase_experiments.md.

Честные ограничения

1. Частота 0.057/нед = 1 раз в 2–3 месяца. Цель кейса — 1–2/нед. Параметрические эксперименты (5 вариантов) подтвердили: нет конфигурации, которая даёт ≥ 0.10 /нед с CI > 1.0 на KGS. Это фундаментальный trade-off данного класса индикаторов на этих данных.

2. N=4–8 OOT сигналов на коридор. OOT период — 14 месяцев (01.07.2025–05.09.2026). При частоте 0.057/нед это ~3–4 события на коридор. CI bootstrap технически корректен, но на малых N — широкий. Нужен живой пилот.

3. Курс ЦБ ≠ курс исполнения. Бэкtest считается по курсам ЦБ. В приложении курс привязан к поставщикам ликвидности. Разница 10–50 bps возможна. Реальная экономия в пилоте может отличаться.

4. KZT и UZS не работают. KZT — управляемый курс (NBK interventions), нет sharp strengthening episodes → 0 OOT сигналов. UZS — lift есть, CI не проходит.

Чеклист для ревью (для людей и LLM)

Каждый пункт — конкретная проверка с указанием файла и строки.

- ☐ **Нет lookahead в индикаторе:** `src/indicators/log_return_percentile.py:41` — первая строка `compute()` вызывает `self._filter(df, corridor, cutoff_date)`. Весь расчёт после этой строки работает только на отфильтрованных данных.
- ☐ **Pathwise hit definition:** `src/backtest/metrics.py` — функция `hit_rate_at_h()`. Проверить: берётся `min(future_slice)`, а не `future_slice[-1]`. Иначе lift завышен — клиент «выиграл» только если курс хоть раз был лучше, а не в конце.
- ☐ **bps формула:** `src/backtest/metrics.py` — функция `bps_by_horizon()`. Формула: $(\text{mean}(\text{rate}[t+1..t+h]) - \text{rate}[t]) / \text{rate}[t] \times 10\,000$. Положительное = день сигнала *дешевле* среднего курса в следующие h дней. Отрицательное = клиент переплатил. Проверить знак для c2 (ожидаем +) и c0 (ожидаем -).
- ☐ **Lift baseline:** `src/backtest/metrics.py` — функция `lift_over_random()`. Baseline = hit rate случайного дня на том же торговом календаре. Проверить: baseline не считается на всём ряду, а только на trading days тестового окна.
- ☐ **CI bootstrap параметры:** `src/backtest/metrics.py` — функция `lift_confidence_interval()`. Параметры: блоки 90 дней, circular, 2000 resamples, 90% confidence. Проверить: блоки не перекрываются с IS при circular resample.
- ☐ **OOT split чистый:** `src/backtest/engine.py` — константа `OOT_START`. Должна быть = `2025-07-01`. Параметры индикатора в `scripts/run_oot_validation.py` не менялись после этой даты (`git log` для проверки).
- ☐ **Compliance push-текстов:** `src/texts/templates.py` — запрещённые паттерны («скоро», «успейте», «гарантируем», «вырастет», «заработайте») должны триггерить исключение при генерации push-текста.
- ☐ **Тесты проходят:** запустить `make test` → ожидается 30 passed, 0 failed.

Воспроизвести результаты

```
git clone https://github.com/davadevi/fx-signal-layer.git
cd fx-signal-layer
pip install -r requirements.txt

# Данные
python -m src.data.download
python -m src.data.normalize

# Воспроизвести главный результат (~15 мин)
PYTHONPATH=. python scripts/run_oot_validation.py
# → reports/validation/oout_validation_{date}.json

# Тесты
make test # ожидается: 30 passed

# Сигналы на сегодня
PYTHONPATH=. python -m src.pipeline.run --cutoff-date $(date +%Y-%m-%d)
```

Файловая карта

Файл	Что там
src/indicators/log_return_percentile.py	Основной индикатор — весь расчёт
src/backtest/engine.py	Walk-forward движок, BacktestResult, OOT_START
src/backtest/metrics.py	hit_rate, lift, CI bootstrap, bps_by_horizon
src/pipeline/signals.py	generate_signals(), cooldown, compliance filter
src/texts/templates.py	Push-тексты + compliance validator
scripts/run_oout_validation.py	Скрипт воспроизведения + c0 сравнение
reports/validation/oout_validation_2026-09-04.json	Сырые числа (c2, c0, c1, t25, t30, w30)
docs/experiments/frequency_increase_experiments.md	Параметрические эксперименты по частоте
tests/unit/test_no_lookahead.py	

Unit-тесты на отсутствие
lookahead

FX Signal Layer · Alfa-Bank Hackathon · 05.09.2026 · Числа: reports/validation/
oot_validation_2026-09-04.json · Для вопросов: /review или открыть issue в репо